VIDEO SCRIPTS — LAKSH AHUJA (24BDS0212)

AI-Driven Edge–Cloud Network for Cyclone Prediction and Resilient Ocean Telemetry

> *How to use: read the plain text aloud at a relaxed pace (about 135 words per minute). Lines marked SHOW are screen or camera cues, not spoken. Fill in your team number. Keep your face visible.*

1. Review 1 — Problem Identification & Proposal

Date: 24/8/2026 **Spoken length:** 648 words ≈ 4.8 minutes **File name:** 24BDS0212_LakshAhuja_R1_Video

0:00–0:15 Introduction

Hello, I am Laksh Ahuja, register number 24BDS0212, team number ____. Our project is “AI-Driven Edge–Cloud Network for Cyclone Prediction and Resilient Ocean Telemetry”.

> *SHOW: your face, then the title slide.*

0:15–1:30 Problem and evidence

Our problem is this: when a cyclone approaches, ocean buoys and coastal stations have to send their readings over slow and unreliable links, and the readings that matter most get delayed or lost exactly when the weather is worst.

Why does it matter? India has a coastline of about seven and a half thousand kilometres, and the Bay of Bengal is one of the most cyclone-prone regions in the world. The 1999 Odisha super cyclone alone killed about ten thousand people, according to the India Meteorological Department. The WMO's fifty-year disaster atlas lists tropical cyclones among the biggest causes of deaths and economic losses from weather.

The people affected are coastal communities, fishers, and the disaster-management teams who order evacuations. If this is not solved, warning systems show old or missing data during the most critical hours.

There is also a bandwidth problem: a single satellite image with four bands is about two megabytes, which would block a slow satellite link for a long time.

> *SHOW: the IMD / WMO sources and one cyclone satellite image.*

1:30–2:45 Literature survey and gap

We surveyed fifteen papers. In networking, DiffServ gives packets fixed priority classes, and RED drops packets early when queues fill up, but the priorities are static and the drops do not care whether a packet is critical.

For satellite images, the Dvorak technique and the Advanced Dvorak Technique estimate cyclone strength from cloud patterns, and newer work uses deep CNNs, but all of them run centrally on full images.

For missing data, models like GRU-D and BRITS fill gaps in time series very well, but offline, on complete datasets, using heavy frameworks.

So the gap is: nobody combines all of these in one network — priority that follows the live cyclone risk, processing images at the edge, recovering lost readings in real time, and feeding the alert back to the

network. That is what our project fills.

> *SHOW: the literature table with Author / Method / Result / Limitation.*

2:45–4:15 Proposed solution and feasibility

Our solution is a coding and modelling project: a three-tier edge–cloud system. Sensor nodes send small packets to an edge gateway. The gateway sorts them by importance and sends the most important first to a central prediction server, which repairs missing data, calculates a cyclone risk score and shows it on a live dashboard.

We chose coding and simulation first, not fabrication, because we cannot put buoys in the ocean. Software lets us prove the mechanism, measure it, and repeat experiments. Low-cost ESP32 sensor nodes can be added later as hardware.

The system has four key ideas: process big satellite images at the edge and send only a small summary; send critical cyclone data first when bandwidth is short; estimate lost readings instead of leaving gaps; and send the alert back to the network so it can react.

> *SHOW: the block diagram — sensors, edge gateway, central server, dashboard.*

On feasibility: we already know Python and basic networking. We need to learn FastAPI for the server, time-series models like LSTM, and Cisco Packet Tracer for the network topology. All tools are free and open source, and the project runs on our own laptops.

The main risks are: we do not have real buoy data, so we will simulate it; real network loss is hard to reproduce, so we will emulate it; and the AI parts may take longer than planned.

4:15–5:00 My role and timeline

My role in the team is the central server and the dashboard, the traffic-priority scheduler, the cyclone simulation model, and testing. Anvay works on the edge gateway and edge AI, and Sreehari on the sensors, hardware and data recovery.

Our plan up to Review 2: in week one, define the message format and build the Packet Tracer topology; in week two, get packets flowing over UDP with simulated loss; in week three, build the server and dashboard, so that by Review 2 we have a working end-to-end prototype. Thank you.

> *SHOW: the team role table and the weekly timeline.*

2. Review 2 — Design, Development & Progress

Date: 14/9/2026 **Spoken length:** 677 words ≈ 5.0 minutes **File name:** 24BDS0212_LakshAhuja_R2_Video

0:00–0:15 Introduction

Hello, I am Laksh Ahuja, register number 24BDS0212, team number ____. Our project is “AI-Driven Edge–Cloud Network for Cyclone Prediction and Resilient Ocean Telemetry”.

> *SHOW: your face, then the architecture diagram.*

0:15–1:55 Design walkthrough

Here is our system design. There are three layers. First, the sensor layer: buoys and radars that send compact JSON packets with a node name, a sequence number, wind, temperature and pressure, over UDP.

Second, the edge gateway. It checks every packet and marks it CRITICAL if wind is sixty kilometres per hour or more, or pressure is nine hundred and ninety hectopascals or less. Critical packets go into a priority queue and are forwarded first.

Third, the central server, which is my part. Its flow is: receive a packet, check the sequence number, and if numbers are missing — say packet ten arrives and then packet thirteen — it fills in eleven and twelve and marks them as reconstructed. Then it calculates a risk score from zero to one hundred, using wind and pressure, and serves the results through a REST API to the dashboard.

> *SHOW: the flow diagram and central_server.py.*

For the design choices: we used UDP and not TCP, because TCP's retransmissions would hide exactly the packet loss we are trying to handle. We used a priority queue instead of fixed bandwidth limits, because cyclone urgency is unpredictable. We start with linear interpolation for gaps, as a simple baseline to compare a trained LSTM against later. And we chose FastAPI over Flask, because it handles a background UDP listener and web requests together, and it generates API documentation automatically.

1:55–3:35 Live progress — my work

This is real, running code, not a plan. We have four Python modules committed to GitHub, plus the network topology in Cisco Packet Tracer.

> *SHOW: the GitHub repository and commit history.*

Now I start the central server, the gateway and two simulated sensors. In a twelve-second test run, the server received sixty-six packets, reconstructed eleven that were lost, and classified two as critical once the wind crossed sixty. The predict endpoint reported an overall risk of one hundred and the alert turned on.

> *SHOW: terminals running, then /predict and the dashboard in the browser.*

The dashboard refreshes every two seconds. It shows the latest reading from each node, its risk, whether it is CRITICAL, and reconstructed values highlighted in yellow, so you can see exactly which data was estimated.

In Packet Tracer we built the routed topology: a core router, ocean, coastal and satellite routers with

OSPF, and a data-centre switch. A ping from BUOY-01 to the prediction server succeeds end to end.

> *SHOW: the Packet Tracer topology and the successful ping.*

We are about half way. What remains is the LSTM model, a machine-learning classifier for priority, edge processing of satellite images, a live WebSocket dashboard, and automated tests.

3:35–4:20 A problem and how I solved it

One real problem: at first, the server's predict endpoint returned empty data, because the UDP listener thread had not started before the sensors began sending. I found it by checking the server log, which showed the listener starting late. I fixed it by starting the listener explicitly in FastAPI's startup event, and after that the data arrived immediately.

A second problem: when we started the three processes from separate terminals, background processes lost their working directory and failed silently. We solved it by launching everything from one script.

We also found a real design limit: a gap can only be repaired after the next packet arrives, because only then do we know a packet was missing. For Review 3 I plan to add real-time predictions for packets that are overdue.

4:20–5:00 Cost and my contribution

Our cost is still zero rupees for development, because Python, FastAPI and uvicorn are all free. Deploying the server later on a small cloud machine would cost about five to ten dollars a month, and low-cost sensor hardware would add about eight hundred rupees per node.

Socially, coastal and island communities usually have the weakest connectivity, so a network that keeps working when links degrade matters for fair disaster response. Environmentally, development needs no new hardware, and sending less data also means less radio energy at remote stations.

My contribution so far is the central server, the reconstruction logic, the risk score, the REST API and the dashboard, plus our test runs. Thank you.

> *SHOW: your commits and the files you wrote.*

3. Review 3 — Final Prototype, Report & Viva

Date: 9/10/2026 **Spoken length:** 601 words ≈ 4.5 minutes **File name:** 24BDS0212_LakshAhuja_R3_Video

0:00–0:15 Introduction

Hello, I am Laksh Ahuja, register number 24BDS0212, team number ____. Our project is “AI-Driven Edge–Cloud Network for Cyclone Prediction and Resilient Ocean Telemetry”.

> *SHOW: your face, then the terminal.*

0:15–2:15 Live demonstration (record without cuts)

I will now run the complete system with one command: python run_demo.py. This starts the central server, the edge gateway, three sensor nodes and a satellite ground station.

> *SHOW: run the command, open http://127.0.0.1:8000.*

At the start everything is green: normal conditions, low risk, all traffic is NORMAL. The input is three buoys and radars losing about twelve percent of their packets in bursts, like rain fade at sea.

Now the simulated cyclone is approaching. You can see pressure falling and wind rising in the charts. Packets are turning CRITICAL, and some are marked by the machine-learning classifier even before the wind crosses sixty.

Now the banner turns red — cyclone alert. The fused risk combines the sensors, the satellite and the forecast. The yellow dots in the charts are packets that were lost and rebuilt by our LSTM model, and the hollow dots are real-time predictions while a packet is late.

On the right is the satellite panel. Each image is about two megabytes, but the ground station analyses it and sends only about one kilobyte — that is ninety-nine point nine five percent less data.

> *SHOW: close-up of the QoS table.*

This table is the part I built in the gateway: critical packets wait only a few hundred milliseconds, while normal and unimportant packets wait longer. That is the important data going first.

2:15–4:00 Test results and failures fixed

My benchmark was: under congestion, ninety-five percent of critical packets must arrive within one second, with at least ninety-nine percent delivered. We tested it with three hundred seconds of traffic on links of one to two kilobytes per second.

With our scheduler, ninety-five percent of critical packets arrived within about three hundred and fifty milliseconds. With a normal first-come-first-served queue, it took forty-nine seconds. On the tightest link, the old method lost twenty-three percent of critical packets; ours lost none. And when the link has spare capacity, both behave the same, so nothing is wasted.

> *SHOW: the QoS results chart.*

In a seventy-five-second live run, the alert turned on at about forty-five seconds, the server rebuilt twenty-six lost readings, and all twenty-eight automated tests pass.

Two things failed along the way. First, my evaluation hung forever, because two scheduler functions used slightly different rules; I made them share one rule and added a test for it. Second, I first capped normal traffic at fifty percent during an alert, and the tests showed this actually made critical packets

slower. I replaced it with a small reserved amount of bandwidth for critical packets, and the delay went back to zero.

4:00–4:30 Comparison with literature

Compared with DiffServ, our priority follows the live cyclone risk instead of fixed classes. Compared with RED, we only drop unimportant data, and only when the link is congested. Compared with edge-computing papers that describe the idea, we built and measured a working three-tier prototype. Where we did not improve: our results come from simulated data, and our scheduler has only been tested on one gateway.

> *SHOW: the comparison table.*

4:30–5:00 My contribution and scalability

My contribution is the central server, the live dashboard, the QoS scheduler, the cyclone model, the demo launcher and the test suite.

To deploy this beyond a prototype, we would run the server on the cloud, gateways on Raspberry Pis at coastal sites, add security to the API, and test it with real buoy data from INCOIS or NOAA. At scale, each sensor node costs about eight hundred rupees and the server a few hundred rupees a month, which makes dense coastal networks affordable. Thank you.

> *SHOW: your files and commits.*